

Sistema Integral de Detección de Infracciones y Estimación de Velocidades Mediante Visión por Computadora

Martín Bianchi
Departamento de Ingeniería
Universidad de San Andrés
Victoria, Argentina
martinbianchi@udesa.edu.ar

Agustín Amblard
Departamento de Ingeniería
Universidad de San Andrés
Victoria, Argentina
aamblard@udesa.edu.ar

Federico Gutman
Departamento de Ingeniería
Universidad de San Andrés
Victoria, Argentina
fgutman@udesa.edu.ar

Abstract—Este trabajo presenta un sistema integral de fiscalización de tránsito basado únicamente en video, capaz de estimar velocidades vehiculares, leer patentes y clasificar infracciones a partir de una cámara fija. El pipeline combina un detector YOLOv11-small con seguimiento ByteTrack, proyección geométrica al plano real (homografías por carril y auto-calibración con AnyCalib) y un módulo específico de reconocimiento de patentes mediante FastPlate-OCR con votación temporal por vehículo.

El sistema se entrena y evalúa sobre UA-DETRAC y Vehicle-DSM, utilizando mediciones de radar como *ground truth*. Los resultados muestran que el método de homografías por carril, calibrado con radar, alcanza errores de velocidad de sólo unos pocos km/h y que el detector de patentes proporciona recortes suficientes para una lectura confiable. A nivel de sistema integrado, la clasificación de infracciones logra alta precisión y *recall*, indicando que una solución basada sólo en cámaras puede aproximar la calidad de un radar aprovechando la infraestructura ya instalada en las ciudades.

Index Terms—traffic enforcement, speed estimation, license plate recognition, homography, computer vision, deep learning

I. INTRODUCCIÓN

La detección automática de infracciones de tránsito es un componente esencial en la modernización de las ciudades y en la mejora de la seguridad vial. Si bien los sistemas basados en radares ofrecen mediciones precisas de velocidad, su instalación y mantenimiento implican costos elevados y limitan su despliegue a un número reducido de intersecciones. En contraste, la gran mayoría de las ciudades ya cuenta con una extensa red de cámaras de videovigilancia, lo que abre la posibilidad de desarrollar soluciones basadas únicamente en video que permitan ampliar la cobertura sin necesidad de infraestructura adicional.

En este trabajo se presenta un sistema integral de detección de infracciones basado exclusivamente en una cámara convencional, capaz de estimar velocidades vehiculares y detectar potenciales violaciones de tránsito. El sistema integra tres componentes principales: un módulo de detección y seguimiento de vehículos entrenado y adaptado al escenario de interés, un estimador de velocidad basado en homografías

por carril y técnicas de calibración geométrica automática, y un componente de lectura de patentes que combina detección basada en bordes con *optical character recognition* (OCR) especializado.

El sistema se evalúa sobre un conjunto de datos compuesto por secuencias reales de tránsito con mediciones de radar como *ground truth*. Se presentan métricas de desempeño tanto para detección y *tracking* como para estimación de velocidad y reconocimiento de patentes, analizando los casos de éxito y las principales limitaciones del enfoque.

Los resultados muestran que es posible obtener mediciones comparables a las de un radar utilizando únicamente video, lo que demuestra el potencial de estas técnicas para reducir costos, aprovechar la infraestructura urbana existente y habilitar soluciones escalables para la fiscalización automática del tránsito.

II. CONJUNTO DE DATOS Y CARACTERÍSTICAS

A. UA-DETRAC

El conjunto de datos UA-DETRAC es un *benchmark* ampliamente utilizado para tareas de detección de vehículos (*vehicle detection*) y seguimiento múltiple (*multi-object tracking*) en escenarios urbanos. El conjunto consta de más de 140 000 *frames* de videos de tráfico del mundo real capturados en 24 ubicaciones diferentes, incluidas autopistas e intersecciones. Los videos presentan condiciones ambientales variadas, incluyendo cambios de iluminación, clima (nublado, lluvia, niebla) y densidad vehicular.

Cada cuadro contiene anotaciones cuadro a cuadro (*frame-by-frame*) con cajas delimitadoras (*bounding boxes*) y categorías básicas de vehículo. La diversidad intra-escena y la presencia frecuente de oclusiones hacen que este conjunto de datos sea especialmente adecuado para entrenar detectores robustos y modelos de *tracking* resilientes.

En este trabajo se utiliza UA-DETRAC como conjunto de datos base para entrenar y evaluar un detector YOLOv11-small, que luego se integró al *pipeline* de detección y seguimiento de vehículos. El objetivo del uso de este conjunto fue obtener un modelo inicial con fuerte capacidad de

generalización antes del ajuste fino sobre datos específicos del escenario de estudio.

B. Vehicle-DSM

El segundo conjunto de datos, *Vehicle-DSM*, propuesto por Luvizon et al., está destinado a tareas de estimación de velocidades vehiculares y reconocimiento de patentes en vías urbanas. Este conjunto consta de aproximadamente cinco horas de grabación en resolución *Full HD* (1920×1080 píxeles) a 30 fps, capturadas mediante una cámara cenital que abarca tres carriles adyacentes. Los videos presentan una clasificación detallada de condiciones ambientales y de grabación adversas, incluyendo lluvia, desenfoque por movimiento (*motion blur*), ruido artificial y variaciones severas de iluminación.

Además de las anotaciones de patentes, el dataset incluye regiones predefinidas para cada carril, representadas por rectángulos o polígonos en coordenadas de imagen. Estas regiones se utilizaron para construir homografías por carril que permiten proyectar la base del *bounding box* de cada vehículo al plano métrico. De esta manera, se dispone de correspondencias píxel-mundo reales necesarias para estimar velocidades comparables con las mediciones de radar.

Las etiquetas del conjunto contienen también mediciones de velocidad real obtenidas mediante un sistema de detectores de bucle inductivo calibrado por el Instituto Nacional de Metrología, Calidad y Tecnología de Brasil (INMETRO), lo que permite evaluar con precisión el desempeño de los métodos de estimación de velocidad.

En este trabajo se utiliza primero una pequeña muestra de este conjunto de datos para realizar un *fine-tuning* del modelo YOLOv11 previamente entrenado con UA-DETRAC, adaptándolo a la perspectiva cenital del escenario. Luego, las regiones de carril y las mediciones de radar se emplean para desarrollar y evaluar los módulos de proyección al plano real, estimación de velocidades y lectura de patentes.

Es relevante mencionar que las *bounding boxes* del conjunto de datos corresponden a las patentes y no a los vehículos, dificultando el proceso de *fine-tuning*. Para solucionar este problema, se colocaron manualmente las etiquetas sobre los vehículos utilizando Roboflow.

III. METODOLOGÍA

El sistema propuesto toma como entrada un flujo de video proveniente de una única cámara fija y produce, para cada vehículo, su trayectoria, su velocidad estimada y la lectura de la patente. El pipeline consta de tres módulos principales: detección y tracking de vehículos, estimación de velocidad mediante proyección geométrica y calibración, y detección y reconocimiento de patentes. Cada componente genera información que alimenta al siguiente, conformando un flujo automatizado.

A. Detección de vehículos

Para la detección utilizamos **YOLOv11-small**, una arquitectura moderna de detección *anchor-free* optimizada para entornos donde se requiere inferencia en tiempo real. El modelo

fue inicialmente entrenado sobre el dataset UA-DETRAC, el cual se dividió tres conjuntos: un conjunto de entrenamiento con el 70% de los datos, un conjunto de validación con el 20% de los datos, y un conjunto de prueba con el 10% de los datos. Un detalle a tener en cuenta es que como varios *frames* distintos pueden pertenecer al mismo video, nos aseguramos de no mezclar frames de un mismo video en distintos conjuntos para evitar *data leakage*. Este entrenamiento permitió obtener un modelo de detección de vehículos robusto frente a variaciones de iluminación, oclusiones parciales y distintos tipos de vehículos. Una vez obtenidos los pesos del modelo entrenado con el primer dataset, se congelaron las capas pertenecientes al *backbone* y se realizó un *fine-tuning* sobre un conjunto reducido de datos proveniente del Vehicle-DSM, con el objetivo de adaptar el detector a la perspectiva de nuestra cámara.

El seguimiento temporal de los vehículos frame a frame se implementó utilizando ByteTrack. La ventaja de este algoritmo además de su precisión, es la poca carga computacional que conlleva respecto a otros métodos, convirtiéndolo en una buena opción para la detección y seguimiento en tiempo real.

B. Proyección al plano y calibración geométrica

Para transformar las trayectorias en coordenadas del mundo real se utilizan dos enfoques complementarios: un método basado en homografías por carril calibradas con radar y dos variantes de calibración automática mediante AnyCalib.

1) *Homografías por carril calibradas con radar*: Para un subconjunto de videos que incluyen mediciones reales de radar, se definió para cada carril una *trampa de velocidad* rectangular en la imagen, apoyada sobre la calzada y alineada con el sentido de circulación. Estas regiones no fueron dibujadas por nosotros: forman parte del propio conjunto de datos *Vehicle-DSM*, que provee para cada carril un rectángulo (o polígono equivalente) ya anotado en coordenadas de imagen. A cada uno de estos rectángulos en píxeles se le asocia un rectángulo en el plano del mundo real con dimensiones conocidas (por ejemplo, 2 m de ancho por 4.8 m de largo). Esto genera las correspondencias punto a punto necesarias entre la imagen y el plano de la calzada.

Para cada carril ℓ se estima una homografía $H_\ell \in \mathbb{R}^{3 \times 3}$ que mapea coordenadas de imagen (x, y) a coordenadas métricas (X, Y) en el plano de la calzada.

Dado un vehículo v y su *bounding box* en el cuadro n , se considera el punto en la base de la caja (aproximando el contacto con el piso) con coordenadas de imagen

$$\mathbf{p}_{v,n} = (x_{v,n}, y_{v,n}).$$

Este punto se proyecta al plano real usando la homografía del carril correspondiente:

$$\mathbf{P}_{v,n} = (X_{v,n}, Y_{v,n}) = \Pi(H_\ell, \mathbf{p}_{v,n}), \quad (1)$$

donde $\Pi(\cdot)$ denota la transformación proyectiva a coordenadas homogéneas seguida de la normalización a coordenadas cartesianas. Solo se consideran los cuadros en los que $\mathbf{p}_{v,n}$ se

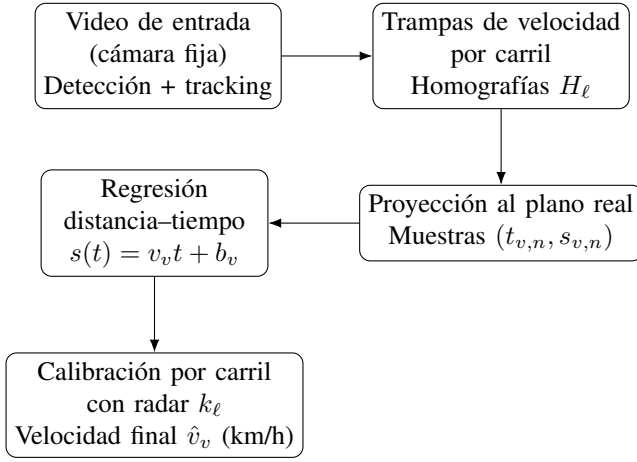


Fig. 1. Esquema del método de estimación de velocidad basado en homografías por carril y calibración con radar.

encuentra dentro del polígono de la trampa de velocidad definida para ese carril.

Sobre el plano métrico, se define un eje longitudinal asociado al carril ℓ , representado por un vector unitario $\mathbf{u}_\ell$ que apunta en el sentido de circulación. La distancia recorrida por el vehículo sobre ese eje en el cuadro n se obtiene proyectando la posición al eje del carril:

$$s_{v,n} = \mathbf{u}_\ell^\top (\mathbf{P}_{v,n} - \mathbf{P}_{\ell,0}), \quad (2)$$

donde $\mathbf{P}_{\ell,0}$ es un punto de referencia fijo sobre la trampa de velocidad (por ejemplo, uno de sus vértices en el plano real).

Los tiempos asociados a cada muestra se calculan a partir del número de cuadro y la frecuencia de muestreo del video. El resultado de este proceso es, para cada vehículo que atraviesa la trampa, una serie de pares $\{(t_{v,n}, s_{v,n})\}$ en coordenadas métricas, que describe la trayectoria unidimensional del vehículo dentro de la región de medición.

2) *Calibración automática con AnyCalib*: AnyCalib estima la geometría de la cámara sin requerir puntos de referencia físicos. A partir de una sola imagen, el método combina redes profundas (DINOv2 y Light-DPT) para predecir un campo de ángulos de visión (*FoV field*) por píxel. Este campo se ajusta mediante optimización convexa y se interpreta como un conjunto de rayos que salen del centro óptico; a partir de ellos, el modelo recupera en forma cerrada los parámetros intrínsecos de la cámara resolviendo un sistema lineal $A\mathbf{x} = \mathbf{b}$.

En nuestro caso, utilizamos estos rayos para definir un plano de calzada en el espacio 3D a partir de puntos de referencia sobre la calzada que ya vienen provistos por el propio *dataset*, junto con las velocidades de radar de *ground truth*. Sobre este plano proyectamos las detecciones de los vehículos, pero restringiéndonos a las *bounding boxes* que pasan por una región central de la imagen para evitar zonas con mayor distorsión o perspectivas extremas. Esto produce trayectorias coherentes en un sistema de unidades arbitrario que luego transformamos a metros reales utilizando un *prior* geométrico del mundo físico (por ejemplo, el ancho promedio

de un vehículo o la distancia conocida entre dos puntos de la escena).

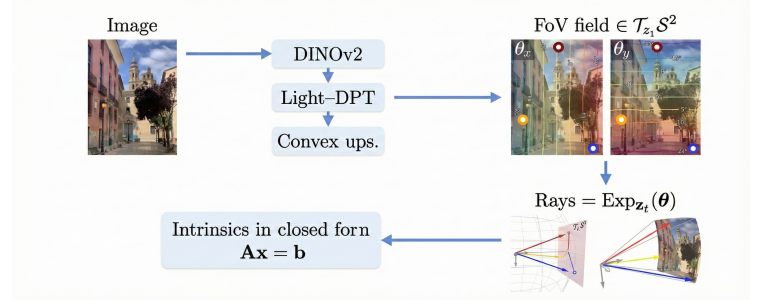


Fig. 2. Esquema simplificado del pipeline de AnyCalib.

C. Estimación de velocidad

Una vez proyectadas las posiciones al plano real (ya sea mediante homografías por carril o mediante AnyCalib), la velocidad se obtiene a partir de la evolución temporal de la distancia recorrida dentro de la región de medición. Para reducir el impacto del ruido en las detecciones, se considera el conjunto de puntos $\{(t_{v,n}, s_{v,n})\}$, donde $s_{v,n}$ se obtiene de (2) y $t_{v,n}$ es el instante temporal del cuadro, y se ajusta un modelo lineal de la forma:

$$s_{v,n} \approx v_v t_{v,n} + b_v, \quad (3)$$

donde v_v representa la velocidad promedio del vehículo dentro de la trampa y b_v es un término de desplazamiento inicial. La estimación de v_v se obtiene mediante mínimos cuadrados sobre todos los puntos de la trayectoria, lo que resulta más robusto que el cociente simple entre dos cuadros individuales.

En el caso del método basado en homografías por carril, la velocidad estimada v_v ya se encuentra en unidades métricas (m/s) y se transforma a km/h como

$$v_v^{(\text{km/h})} = 3.6 v_v. \quad (4)$$

Sin embargo, incluso con una buena calibración geométrica, pueden aparecer sesgos sistemáticos *por carril* debido a pequeñas imprecisiones en la ubicación de la trampa, en la homografía o en la tasa de cuadros reales del video. Para corregir estos efectos, se utiliza para cada carril ℓ un subconjunto de vehículos que cuentan con medición de radar $v_v^{(\text{radar})}$. Sobre ese subconjunto se aprende un *factor de escala por carril* k_ℓ que relaciona la velocidad estimada geoméricamente v_v con la velocidad medida por el radar.

El factor se obtiene mediante una regresión lineal *restringida al origen* (sin término independiente), resolviendo el problema de mínimos cuadrados

$$k_\ell = \arg \min_k \sum_{v \in \mathcal{C}_\ell} (v_v^{(\text{radar})} - k v_v)^2, \quad (5)$$

donde $\mathcal{C}_\ell$ es el conjunto de vehículos del carril ℓ que poseen medición de radar. La solución cerrada del problema es:

$$k_\ell = \frac{\sum_{v \in \mathcal{C}_\ell} v_v^{(\text{radar})} v_v}{\sum_{v \in \mathcal{C}_\ell} v_v^2}. \quad (6)$$

Una vez estimado k_ℓ , la velocidad corregida para cualquier vehículo del mismo carril se obtiene como

$$\hat{v}_v = k_\ell v_v, \quad (7)$$

y finalmente se expresa en km/h aplicando la conversión de (4). Este proceso permite compensar diferencias sistemáticas por carril y alinear las velocidades geoméricamente estimadas con las mediciones reales del radar.

En los modelos basados en AnyCalib, el procedimiento es análogo, pero las posiciones $\mathbf{P}_{v,n}$ se obtienen en un sistema de unidades arbitrario. En este caso, se aprende primero un factor global de escala entre unidades del modelo y metros reales utilizando *priors* geométricos del mundo físico: en un modelo se utiliza que el ancho promedio de un vehículo es de 1.9 m, y en otro se emplea la distancia conocida de 4.4 m entre dos puntos determinados de la escena. Una vez fijada esta escala, se aplica el mismo esquema de regresión distancia-tiempo de (3) y, si es necesario, una calibración adicional por carril mediante (7).

D. Detección de patentes

El módulo de detección de patentes opera sobre los *crops* producidos por el detector de vehículos. Para cada *track* se recorta una región de interés (ROI) centrada en la parte trasera del vehículo y se aplica un detector basado en **bordes verticales**, inspirado en el método de Luvizon et al. [1].

Primero se convierte la ROI a escala de grises y se calcula el gradiente vertical $G_x(x, y)$ mediante un filtro de Sobel horizontal. A partir de ese gradiente se construye una imagen binaria de bordes $E(x, y)$ comparando cada magnitud con el promedio global μ de $|G_x|$, multiplicado por un umbral adimensional τ :

$$E(x, y) = \begin{cases} 1, & \text{si } |G_x(x, y)| > \tau \mu, \\ 0, & \text{en otro caso.} \end{cases} \quad (8)$$

De esta forma se destacan principalmente los trazos verticales de alto contraste típicos de los caracteres de una patente, mientras que se suprimen bordes horizontales y ruido de fondo.

Sobre la imagen $E(x, y)$ se realiza un filtrado morfológico para descartar componentes conexas demasiado pequeñas o demasiado grandes para corresponderse con caracteres individuales. Luego se aplica una dilatación en el eje horizontal para unir trazos vecinos y se agrupan regiones compatibles en función de su posición, ancho, alto y relación de aspecto, obteniendo así una o pocas regiones candidatas por ROI.

Finalmente, para cada vehículo se aplica el detector en múltiples *frames* a lo largo de su trayectoria. Las cajas candidatas obtenidas en los distintos tiempos se fusionan, privilegiando las regiones que aparecen de manera consistente (alto solapamiento espacial y presencia en varios cuadros consecutivos). El resultado es una única *bounding box* por

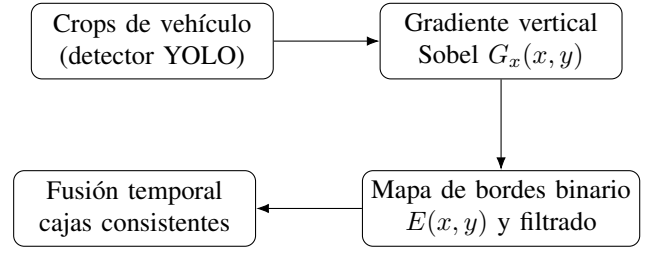


Fig. 3. Esquema del módulo de detección de patentes basado en bordes verticales y fusión temporal por vehículo.

vehículo que encierra la patente y se utiliza como entrada del módulo de lectura.

Para evaluar la calidad de la detección se utilizan dos tipos de métricas. Por un lado, el *intersection over union* (IoU) entre la caja predicha y la caja de *ground truth*. Por otro lado, se definen métricas de cobertura más adaptadas a la lectura de patentes: (i) el porcentaje de detecciones cuyo centro cae dentro de la caja GT (*Centro dentro GT*) y (ii) el porcentaje de casos en los que la caja GT queda completamente incluida dentro de la detección, permitiendo tolerancias de 0, ± 1 y ± 2 píxeles en cada borde (*GT cubierto 0 px, ± 1 px, ± 2 px*). En todos los casos se reportan porcentajes sobre el conjunto de patentes del *dataset*.

E. Lectura de patentes mediante OCR

Una vez localizada la región de la patente, el sistema aplica distintos *pipelines* de reconocimiento óptico de caracteres (OCR, *optical character recognition*) sobre los *crops* normalizados.

Se evaluaron tres motores de OCR:

- **Tesseract**, un OCR clásico de propósito general.
- **EasyOCR**, basado en redes neuronales profundas entrenadas para texto en escenas naturales.
- **FastPlate-OCR**, un OCR ligero especializado en patentes vehiculares, que opera sobre *crops* de patentes y está optimizado para inferencia rápida [2].

Para los tres motores se probaron varias etapas de preprocesamiento de imagen, combinando operaciones morfológicas, filtros de suavizado y realce de contraste:

- **Morpho**: apertura/cierre morfológico para limpiar ruido y reforzar trazos de caracteres.
- **Bilateral + umbral adaptativo**: filtrado bilateral para reducir ruido manteniendo bordes, seguido de binarización local.
- **CLAHE**: ecualización de histograma adaptativa para mejorar contraste en patentes subexpuestas o con reflejos.
- **Umbral adaptativo solo**: binarización local sin filtrado previo.

Cada combinación (*sin preprocesamiento, morpho, bilateral+adaptativo, CLAHE, etc.*) se aplicó antes del OCR correspondiente, obteniendo distintas variantes de *pipeline* que luego se compararon cuantitativamente en la sección de resultados.

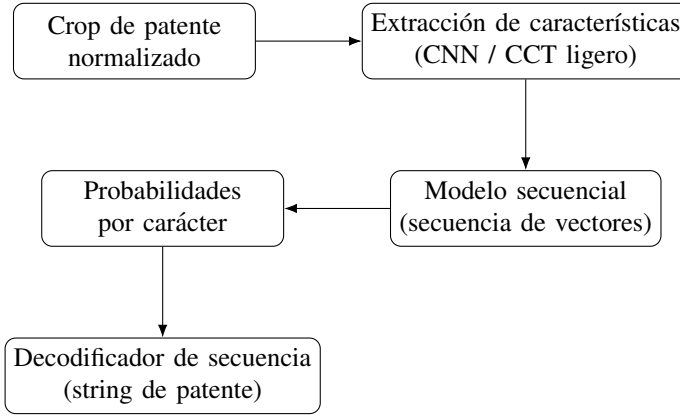


Fig. 4. Esquema conceptual del módulo de lectura de patentes basado en FastPlate-OCR.

En el sistema final se seleccionó **FastPlate-OCR** sin pre-procesamiento adicional, dado que obtuvo el mejor compromiso entre precisión de lectura y simplicidad de implementación. Conceptualmente, el modelo toma como entrada un *crop* normalizado de la patente y lo procesa con una red neuronal ligera (convolucional/transformer compacta) que extrae un mapa de características. Ese mapa se interpreta como una secuencia y se pasa por una cabeza de clasificación que produce, para cada *timestep*, distribuciones de probabilidad sobre un alfabeto de dígitos y letras. Un decodificador convierte luego estas probabilidades en la cadena de caracteres de la patente.

Para consolidar una única lectura por vehículo se explota la redundancia temporal. Cada *track* genera una colección de lecturas $\{p_{v,1}, \dots, p_{v,N_v}\}$ obtenidas en distintos *frames*. La lectura final se calcula mediante votación mayoritaria:

$$\hat{s}_v = \arg \max_s \sum_{n=1}^{N_v} \mathbb{I}(p_{v,n} = s), \quad (9)$$

donde $\hat{s}_v$ es la patente consolidada para el vehículo v y $\mathbb{I}(\cdot)$ es la función indicador. Antes de votar se descartan lecturas que no cumplen un formato válido (longitud esperada y alfabeto permitido), de modo que la votación solo se realiza entre candidatos plausibles.

Para la evaluación cuantitativa del OCR se utilizan tres métricas agregadas: (i) el porcentaje de *crops* cuya salida no es vacía (*non-empty*), (ii) el porcentaje de salidas que cumplen un patrón de patente válida (*looks plate*) y (iii) la longitud promedio de la cadena devuelta (*avg len*), que permite verificar cuán cerca están las lecturas del número de caracteres esperado.

F. Integración de los módulos

Los módulos se integran en un pipeline secuencial que produce, para cada vehículo, su *track_id*, trayectoria proyectada, velocidad estimada y patente. Finalmente, el sistema compara cada velocidad con el límite permitido y genera registros de potenciales infracciones listos para su revisión o almacenamiento.

IV. RESULTADOS

A. Resultados del entrenamiento del YOLOv11

Para establecer una línea base de rendimiento y verificar la capacidad de generalización de la arquitectura YOLOv11, se evaluó el modelo entrenado sobre el conjunto de prueba del conjunto de datos UA-DETRAC. Sobre este conjunto, el modelo demostró un rendimiento robusto alcanzando un **mAP@50** de **0.863** y un **mAP@50-95** de **0.731**, evidenciando su capacidad no solo de detectar los vehículos correctamente, sino también de ajustar las *bounding boxes* con gran precisión. Además, se obtuvo una **precisión** de **0.858** y un **recall** de **0.778**, cifras que indican un comportamiento equilibrado con una baja incidencia de falsos positivos.

Posteriormente, se evaluó el modelo tras el proceso de *fine-tuning* utilizando el conjunto de datos Vehicle-DSM. Los resultados, presentados en la Tabla IV-A, confirman la efectividad de la transferencia de aprendizaje para adaptar el detector a la perspectiva cenital.

Clase	Precisión	Recall	mAP@50-95
Global (All)	0.952	0.979	0.896
Car	1.000	0.938	0.897
Motorcycle	0.998	1.000	0.797
Others	0.857	1.000	0.995

Además, se analizó la eficiencia computacional del modelo para validar su aplicabilidad en escenarios de tiempo real. El modelo YOLOv11-small registró un tiempo de inferencia promedio de 30 ms por cuadro sobre un chip *Apple M1*, lo cual confirma que el módulo de detección es capaz de operar en tiempo real, dejando margen de cómputo suficiente para los procesos subsiguientes.

B. Resultados de estimación de velocidad con homografías por carril

El método basado en trampas de velocidad por carril utiliza directamente las homografías H_ℓ y la calibración con radar para estimar velocidades métricas por vehículo. La evaluación se realiza sobre los vehículos que cuentan con medición de radar, separando los datos en conjuntos de entrenamiento y prueba.

La Fig. 5 muestra la relación entre velocidad estimada y velocidad real para los distintos videos, usando el video 03 como entrenamiento. En todos los casos la nube de puntos se concentra alrededor de la recta identidad $y = x$, con poca dispersión y sin sesgos extremos: las velocidades bajas y medias quedan bien alineadas y solo se observan algunos desvíos aislados en el rango alto. Esto indica que, una vez fijada la geometría por carril, el error principal pasa a ser ruido local de las trayectorias y no tanto problemas de escala.

Entre las variantes evaluadas, la configuración final (trampa por carril, regresión distancia-tiempo y factor de escala k_ℓ aprendido por carril a partir del radar) es la que produce los puntos más pegados a la recta ideal, dando, por ejemplo, un error medio absoluto en el primer video de 1 km/h (MAE $\approx$ 0,77 km/h para $N = 64$ vehículos). No se observan tendencias claras de sobre o subestimación sistemática a lo largo del rango

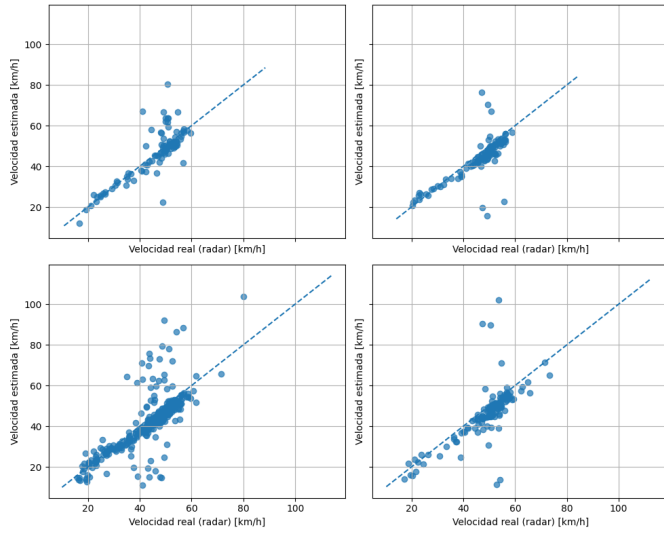


Fig. 5. Velocidad estimada vs velocidad de radar para distintas variantes del método basado en trampas de velocidad por carril. Cada panel corresponde a cada uno de los videos.

de velocidades: los errores son pequeños y aproximadamente simétricos alrededor de la diagonal, lo que confirma que la combinación de homografías por carril y calibración con radar es suficiente para igualar, en la práctica, la precisión del sensor de referencia.

C. Resultados de estimación de velocidad con AnyCalib

Se evaluaron dos variantes del pipeline de estimación de velocidad basadas en AnyCalib. En ambos casos, las trayectorias 2D obtenidas con YOLO+ByteTrack se proyectan al espacio 3D arbitrario del modelo; la única diferencia entre variantes es el *prior de escala* utilizado para convertir esas unidades en metros reales.

1) *Modelo A: prior de ancho del vehículo*: El Modelo A fija la escala física a partir del ancho proyectado del vehículo, asumiendo un ancho real promedio de 1,9 m. Antes de cualquier corrección adicional, el modelo subestima fuertemente las velocidades: las trayectorias quedan comprimidas en el espacio reconstruido y las predicciones caen muy por debajo de la recta $y = x$, como se observa en la Fig. 7. Para compensar esta distorsión, se ajusta un factor global α que iguala la media de las predicciones con la del radar:

$$\alpha = 2.20.$$

En resumen, el prior basado en el ancho del vehículo captura adecuadamente el patrón de movimiento, pero fija una escala inicial demasiado comprimida. El reescalado global α corrige este sesgo y reduce los errores a valores dentro del rango operativo habitual (MAPE $\sim 10\%$).

2) *Modelo B: prior de segmento vertical de 4,4 m*: El Modelo B utiliza un segmento real conocido de 4,4 m alineado con la dirección dominante del movimiento. Proyectando sus extremos en el espacio AnyCalib se obtiene un factor de escala β que se aplica directamente a todas las trayectorias, sin necesidad de ajustes posteriores.

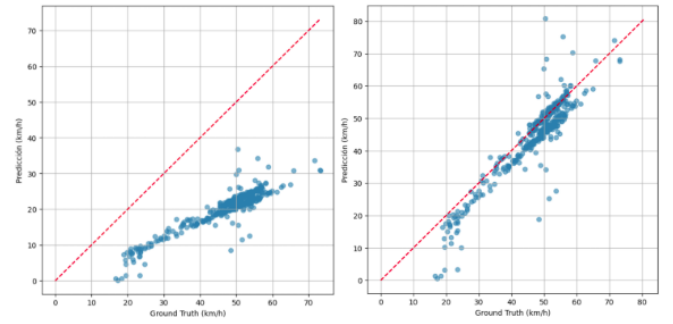


Fig. 6. Comparación entre predicciones sin escalar (izquierda) y con el factor global α (derecha).

Métrica	Entrenamiento	Evaluación	Condición
MAE (km/h)	23.76	26.70	Antes
MedAE (km/h)	25.07	27.86	Antes
RMSE (km/h)	24.52	27.28	Antes
MAPE (%)	56.18	57.74	Antes
R^2	-4.61	-5.95	Antes
MAE (km/h)	3.95	3.69	Después (α)
MedAE (km/h)	3.06	3.41	Después (α)
RMSE (km/h)	6.19	5.45	Después (α)
MAPE (%)	10.83	9.22	Después (α)
R^2	0.642	0.723	Después (α)

TABLE I

MÉTRICAS DEL MODELO A ANTES Y DESPUÉS DEL REESCALADO GLOBAL α .

Este prior es geoméricamente más adecuado porque calibra exactamente el eje relevante para la velocidad. La Fig. 7 muestra que las predicciones se alinean mucho mejor con la recta ideal $y = x$.

Las métricas resultantes:

Métrica	Entrenamiento	Evaluación (Test)
MAE (km/h)	4.49	3.60
MedAE (km/h)	3.06	2.27
RMSE (km/h)	7.36	5.72
MAPE (%)	11.88	8.61
R^2	0.494	0.695
Vehículos evaluados	451	387

TABLE II

MÉTRICAS DEL MODELO B (SEGMENTO VERTICAL DE 4,4 M) PARA LOS CONJUNTOS DE ENTRENAMIENTO Y EVALUACIÓN.

El Modelo B logra un rendimiento más estable y consistente que el Modelo A, con errores relativos por debajo del 10% y sin necesidad de reescalados adicionales. La calibración vertical corrige naturalmente la anisotropía del espacio de AnyCalib y reduce la dependencia de las *bounding boxes*.

D. Resultados de detección de patentes

El módulo de detección se evaluó comparando, para cada patente, la caja predicha con la caja de *ground truth* (GT). Además del IoU promedio, se midieron métricas de cobertura diseñadas para la tarea de lectura: si el centro de la detección cae dentro de la GT y si la caja GT queda completamente cubierta permitiendo pequeñas tolerancias en los bordes. Los valores agregados se muestran en la Tabla III.

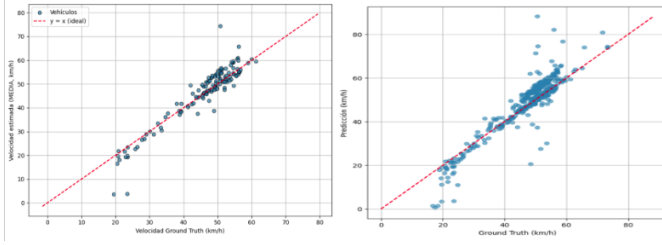


Fig. 7. Relación entre velocidades estimadas y radar para el Modelo B (segmento de 4,4 m).

TABLE III
MÉTRICAS DEL MÓDULO DE DETECCIÓN DE PATENTES.

IoU (todos)	IoU (det.)	Centro en GT [%]	GT 0 px [%]	GT ± 2 px [%]
0.14	0.48	91.2	95.2	95.6

Aunque el IoU medio es modesto, las métricas de cobertura muestran que en la gran mayoría de los casos el centro de la detección cae dentro de la caja GT y casi siempre la patente queda completamente contenida en la caja predicha. Es decir, el detector entrega recortes suficientemente buenos para alimentar al OCR, incluso cuando la superposición no es perfecta según IoU.

La Fig. 8 ilustra esta situación con ejemplos sobre la escena real: las cajas predichas rodean correctamente la región de la patente y permiten una lectura limpia, aun cuando la coincidencia exacta con la anotación GT no sea ideal. Esto confirma que, para esta aplicación, las métricas de cobertura son más representativas de la utilidad del detector que el IoU tomado de forma aislada.

E. Resultados de lectura de patentes mediante OCR

Sobre las cajas generadas por el detector se evaluaron distintas combinaciones de preprocesamiento y motores de *optical character recognition* (OCR): Tesseract, EasyOCR y FastPlate-OCR. La Fig. 9 muestra un ejemplo de los preprocesamientos considerados sobre una misma patente: imagen original en escala de grises, binarización morfológica basada en bordes, filtrado bilateral con umbral adaptativo y ecualización de histograma adaptativa (CLAHE).

Para cada *pipeline* se midieron tres métricas: porcentaje de lecturas no vacías (*non_empty*), porcentaje de cadenas con formato de patente válido (*looks_plate*) y longitud promedio de la predicción (*avg_len*). La Tabla IV resume los principales resultados.

Se observa que FastPlate-OCR sin preprocesamiento adicional es claramente la opción más estable: produce siempre una lectura no vacía, la gran mayoría de las cadenas respeta el formato esperado y la longitud promedio es prácticamente la correcta. Una posible explicación es que la red fue entrenada sobre *crops* de patentes similares a los del dataset, por lo que espera texturas y niveles de ruido “crudos”. Al aplicar preprocesamientos agresivos (binarización dura, filtrado fuerte, ecualización), se altera la distribución de píxeles sobre la que



Fig. 8. Ejemplos de detección de patentes: se comparan las cajas de *ground truth* con las cajas producidas por el detector, mostrando que la región de la patente queda correctamente localizada.



Fig. 9. Ejemplo de preprocesamientos aplicados a una patente: original, binarización morfológica, filtrado bilateral con umbral adaptativo y CLAHE.

el modelo fue entrenado y se pierden detalles finos de los caracteres. Eso se refleja en que las variantes con *morph* o *clahe* empeoran de forma consistente el porcentaje de lecturas válidas.

Los OCR genéricos (EasyOCR y Tesseract) quedan aún por detrás, con más salidas vacías y longitudes menos confiables, lo que sugiere que no están adaptados a texto tan pequeño, borroso y con bajo contraste. Esta diferencia se ve también en los ejemplos cualitativos de la Fig. 10, donde FastPlate-OCR recupera exactamente la secuencia correcta en patentes fuertemente degradadas. Esos casos son justamente donde los OCR genéricos suelen fallar y explican la brecha observada en las métricas de la Tabla IV.

F. Resultados del sistema integrado de detección de infractores

Una vez integrados los módulos de detección y tracking, estimación de velocidad y lectura de patentes, el sistema produce para cada vehículo su velocidad estimada, su matrícula y la clasificación *infracción / no infracción* según el límite de velocidad de la escena.

La Fig. 11 muestra ejemplos cualitativos del pipeline completo. En ambos casos se observa el vehículo en la escena, la caja de la patente, el recorte ampliado y la lectura de FastPlate-OCR, junto con la velocidad estimada y la velocidad medida por el radar. Las diferencias de velocidad son de apenas unos pocos km/h y las patentes leídas coinciden exactamente con el *ground truth*, lo que ilustra que los módulos geométricos y de OCR pueden operar de manera consistente en condiciones reales.

Para cuantificar el desempeño del clasificador de infracciones se construyó una matriz de confusión donde la clase positiva corresponde a *infracción*. La Fig. 12 resume los

TABLE IV
RESULTADOS DE LOS PRINCIPALES *pipelines* DE OCR.

Método	Prep.	nonE [%]	looks [%]	len
FastPlate	none	100.0	93.0	6.93
FastPlate	morph	100.0	61.7	6.60
FastPlate	clahe	100.0	59.1	6.62
EasyOCR	none	96.5	51.3	–
Tesseract	none	80.0	40.0	–



Fig. 10. Ejemplos de patentes en condiciones adversas donde FastPlate-OCR produce una lectura exacta coincidente con la etiqueta de *ground truth*.

resultados. La gran mayoría de los vehículos sin infracción quedan correctamente etiquetados como tales, mientras que solo aparece un falso positivo aislado. Del lado de las infracciones, la mayor parte de los vehículos que superan el límite son detectados, con unos pocos casos que quedan por debajo del umbral del sistema. En términos prácticos, esto significa que el sistema casi no emite multas erróneas y pierde solo una fracción menor de los infractores reales.

A partir de esta matriz se calcularon las métricas agregadas de desempeño, resumidas en la Tabla V. La precisión cercana a 0.97 indica que, entre los vehículos marcados como infracción, casi todos son realmente infractores. El *recall* en torno a 0.92 muestra que el sistema detecta la gran mayoría de las infracciones presentes en el conjunto de evaluación. El puntaje F1 por encima de 0.94 y la exactitud global cercana al 0.96 reflejan un equilibrio sólido entre no generar multas injustificadas y no dejar pasar demasiados casos por encima del límite.

En conjunto, estos resultados muestran que, cuando los módulos de detección, velocidad y OCR se combinan en un único pipeline, el sistema es capaz de identificar infractores con alta confiabilidad usando solo video, manteniendo al mismo tiempo una tasa muy baja de falsos positivos.

G. Discusión y conclusiones

El análisis de los resultados permite extraer algunas conclusiones sobre la viabilidad del sistema. En primer lugar, se evidencia un compromiso entre precisión y facilidad de despliegue. El método de homografías por carril calibradas con radar se establece como la opción superior ($MAE \approx 0.77$ km/h), pero a costo de necesitar mediciones físicas. En contraste, el enfoque con AnyCalib elimina la dependencia de infraestructura previa, aunque introduce una mayor incertidumbre ($MAE \approx 3.60$ km/h).



Fig. 11. Ejemplos del sistema integrado: para cada vehículo se muestra la velocidad real de radar (GT) y la patente leída por FastPlate-OCR.

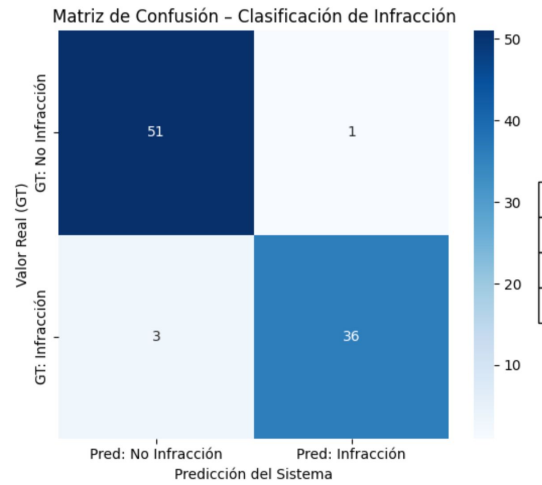


Fig. 12. Matriz de confusión para la clasificación de infracción / no infracción a partir de la velocidad estimada.

En segundo lugar, al comparar las formas de calibrar AnyCalib, se vio que la reconstrucción 3D deforma las distancias de manera desigual. El Modelo A falló porque intentaba calcular el avance del vehículo basándose en su ancho, medida que se distorsiona mucho con la perspectiva. En cambio, el Modelo B demostró que para tener estabilidad, es necesario definir la escala usando una referencia alineada con el movimiento de los autos. Esto logra que el cálculo de velocidad no dependa tanto de si (*bounding box*) varía levemente de tamaño entre cuadros.

TABLE V
MÉTRICAS DEL CLASIFICADOR DE INFRACCIONES.

Métrica	Valor
Precisión	0.973
Recall	0.923
F1-score	0.947
Exactitud	0.956

Finalmente, la integración del sistema prueba que el módulo de lectura de patentes superó ampliamente a las herramientas genéricas al trabajar con imágenes de baja resolución. Esto confirma que una solución basada solo en las cámaras ya instaladas en la ciudad puede acercarse a la fiabilidad de un radar físico reduciendo significativamente los costos de infraestructura.

BIBLIOGRAFÍA

- [1] D. C. Luvizon, B. T. Nassu, and R. Minetto, "A video-based system for vehicle speed measurement in urban roadways," *IEEE Transactions on Intelligent Transportation Systems*, 2016.
- [2] ankandrew, "fast-plate-ocr: Lightweight & fast ocr models for license plate text recognition," <https://github.com/ankandrew/fast-plate-ocr>, 2025, gitHub repository, version 1.0.2, accessed: 2025-12-08.
- [3] L. Wen, D. Du, Z. Cai, Z. Lei, M.-C. Chang, H. Qi, J. Lim, M.-H. Yang, and S. Lyu, "Ua-detrac: A new benchmark and protocol for multi-object detection and tracking," *arXiv preprint arXiv:1511.04136*, 2015, arXiv:1511.04136. [Online]. Available: <https://arxiv.org/abs/1511.04136>

Link al repositorio de GitHub: <https://github.com/martinbianchi0/VisionTrafficGuard>